Introduction to DBMS

Course Overview

BCSC1003: DATABASE MANAGEMENT SYSTEM

Objective: *The objective of the course is to enable students to understand and use a relational database& NoSQL system. Students learn how to design and create a good database.*

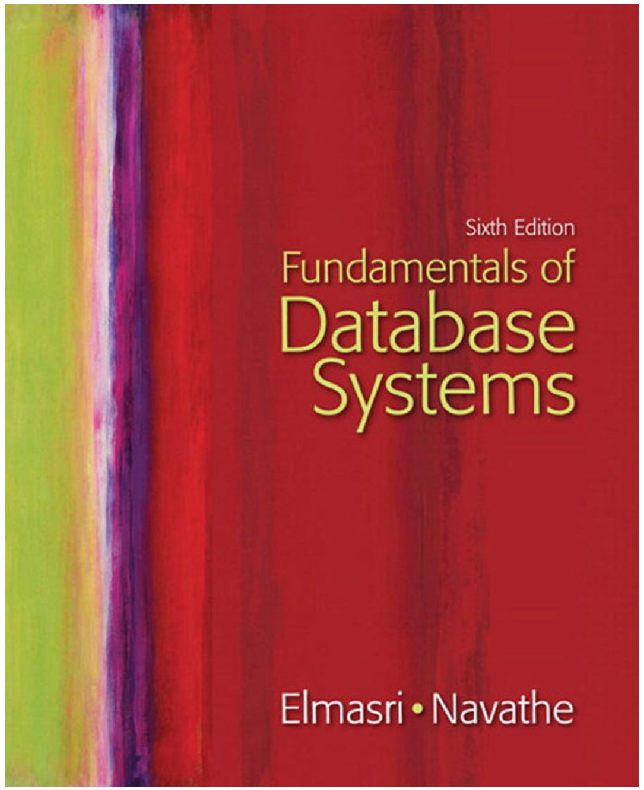
Credits: 03 **L-T-P-J: 3-0-0-0**

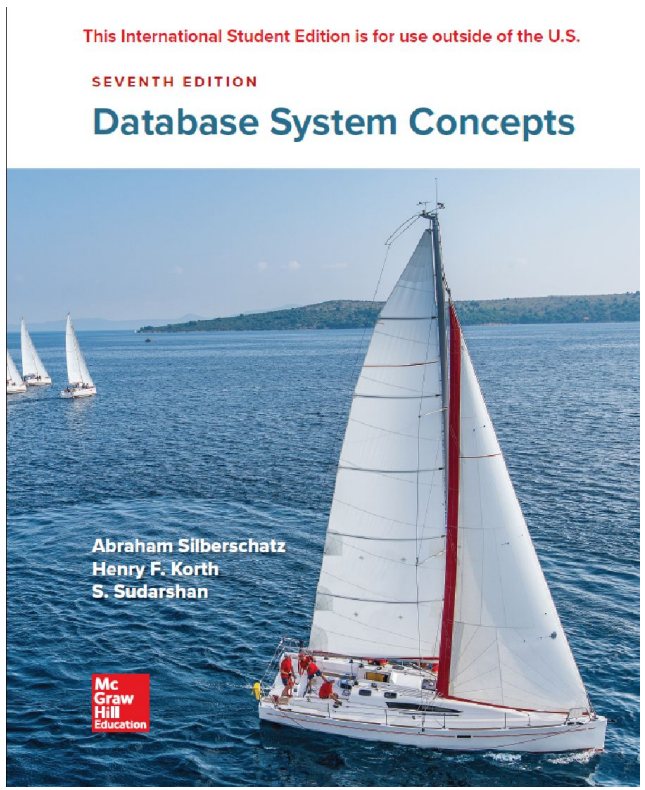
Module No.	Content	Teaching Hours
I	**Introduction:** An Overview of Database Management System, Database System Vs File System, Database System Concept and Architecture, Data Model Schema and Instances, Data Independence, Database Language and Interfaces (DDL, DML, DCL), Database Development Life Cycle (DDLC) with Case Studies. **Data Modeling Using the Entity-Relationship Model:** ER Model Concepts, Notation for ER Diagram, Mapping Constraints, Keys, Specialization, Generalization, Aggregation, Reduction of an ER Diagram to Tables, Extended ER Model. **Relational Data Model and Language:** Relational Data Model Concepts, Integrity Constraints, Entity Integrity, Referential Integrity, Keys Constraints, Primary Key, Foreign Key, Candidate Key, Super Key, Domain Constraints, Relational Algebra. **Database Design & Normalization I:** Functional Dependencies, Canonical Cover, Normal Forms, First, Second, Third Normal Forms, BCNF.	20

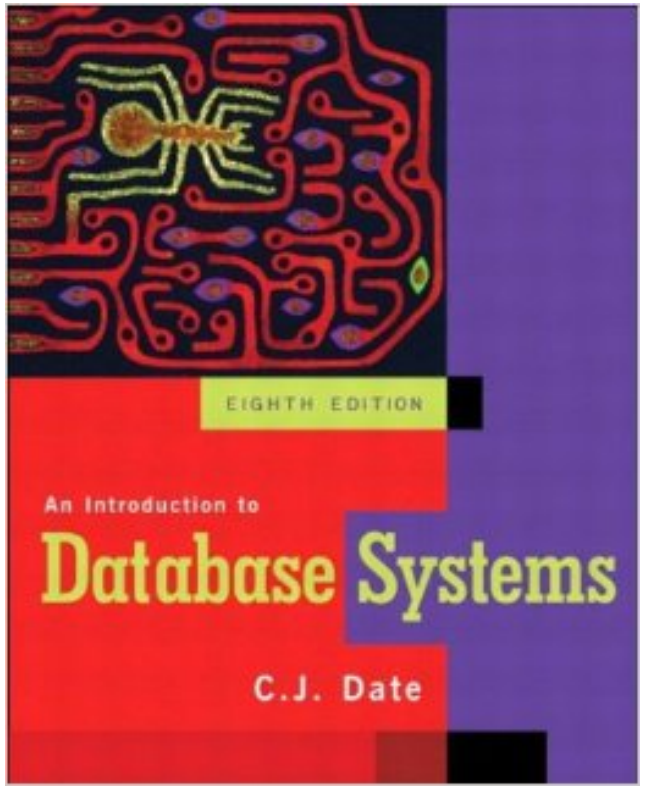
Course Overview

II	**Database Design & Normalization II:** Lossless Join and Dependency Preserving Decomposition, MVD and 4th Normal Form, JD and 5th Normal Form, Inclusion Dependence. **File Organization:** Indexing, Structure of Index files and Types, Dense and Sparse Indexing **Transaction Processing Concept:** Transaction System, Testing of Serializability, Serializability of Schedules, Conflict & View Serializable Schedule, Recoverability, Recovery from Transaction Failures, Log Based Recovery, Deadlock Handling. **Concurrency Control Techniques:** Concurrency Control, Locking Techniques for Concurrency Control, 2PL, Time Stamping Protocols for Concurrency Control, Validation Based Protocol. **Distributed Database:** Introduction of Distributed Database, Data Fragmentation and Replication.	20

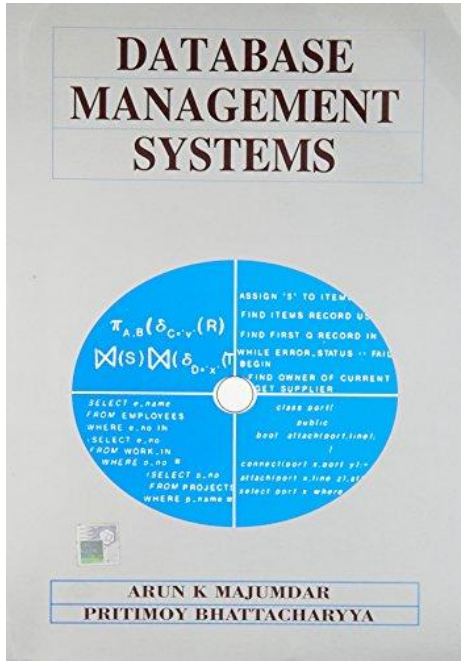
Literature

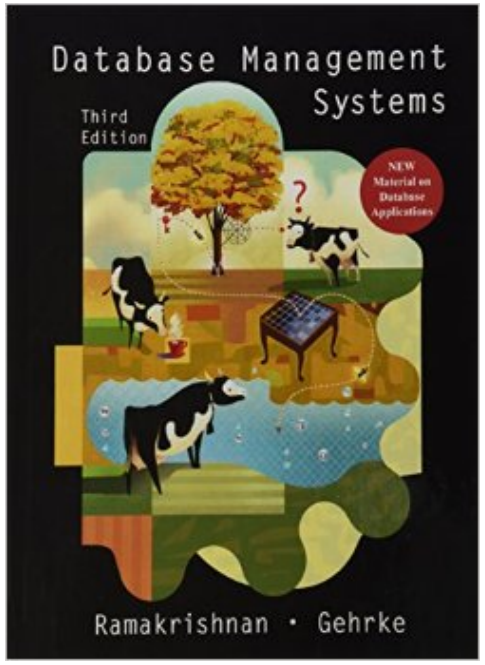

Sixth Edition
Fundamentals of
Database
Systems
Elmasri • Navathe

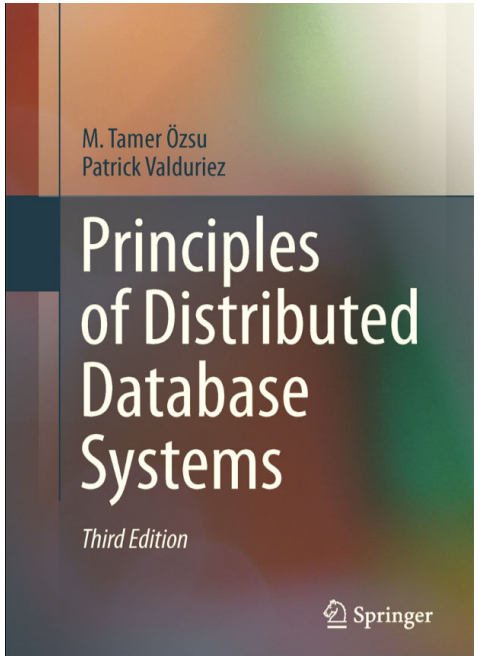

This International Student Edition is for use outside of the U.S.
SEVENTH EDITION
Database System Concepts
Abraham Silberschatz
Henry F. Korth
S. Sudarshan
Mc Graw Hill Education


EIGHTH EDITION
An Introduction to
Database Systems
C.J. Date

Literature


DATABASE
MANAGEMENT
SYSTEMS
ARUN K MAJUMDAR
PRITIMOY BHATTACHARYYA


Database Management
Systems
Third
Edition
NEW
Material on
Database
Applications
Ramakrishnan · Gehrke


M. Tamer Özsu
Patrick Valduriez
Principles
of Distributed
Database
Systems
Third Edition
Springer

Data, Database, DBMS

- **Data:** Known facts that can be recorded and have an implicit meaning; raw

- **Database:** A highly organized, interrelated, and structured set of data about a particular enterprise
  - Controlled by a database management system (DBMS)

- **DBMS:**
  - A collection of programs that enables users to create and maintain a database
  - Set of programs to access the data
  - An environment that is both convenient and efficient to use
  - A software package/system to facilitate the creation and maintenance of a computerized database

Mini-world:

- Some part of the real world about which data is stored in a database. For example, student grades and transcripts at a university.

Database systems are used to manage collections of data that are:

- Highly valuable
- Relatively large
- Accessed by multiple users and applications, often at the same time.

A modern database system is a complex software system whose task is to manage a large, complex collection of data.

Types of Databases and Database Applications

- Traditional applications:
  - Numeric and textual databases
- More recent applications:
  - Multimedia databases
  - Geographic Information Systems (GIS)
  - Data warehouses
  - Mobile databases
  - Real-time and active databases

Recent Developments

- Social Networks started capturing a lot of information about people and about communications among people-posts, tweets, photos, videos in systems such as:
  - Facebook
  - Twitter
  - Linked-In
- All of the above constitutes data
- Search Engines, Google, Bing, Yahoo: collect their own repository of web pages for searching purposes
- New technologies are emerging from the so-called non-SQL, non-database software vendors to manage vast amounts of data generated on the web:
  - **Big data** storage systems involving large clusters of distributed computers
  - **NOSQL** (Non-SQL, Not Only SQL) systems are non-tabular databases
- A large amount of data now resides on the "cloud" which means it is in huge data centers using thousands of machines. Cloud means over the internet.

What a DBMS Facilitates

- DBMS is a general-purpose software system that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications.
- *Define* a particular database in terms of its data types, structures, and constraints
- *Construct* or load the initial database contents
- *Manipulating* the database:
  - Retrieval: Querying, generating reports
  - Modification: Insertions, deletions and updates to its content
  - Accessing the database through Web applications (e.g., GLA Module)
- *Processing* and *sharing* by a set of concurrent users and application programs – yet, keeping all data valid and consistent



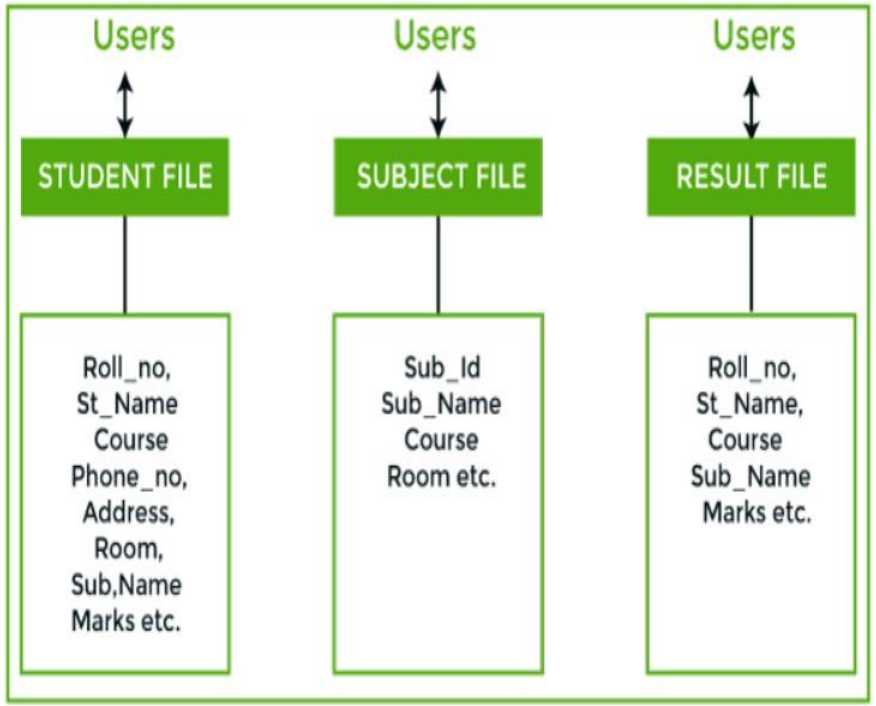
DBMS vs File System

File System

- File based systems were an early attempt to computerize the manual system.
- It is also called a traditional based approach in which a decentralized approach was taken where each department stored and controlled its own data with the help of a data processing specialist.
- The main role of a data processing specialist was to create the necessary computer file structures, and also manage the data within structures and design some application programs that create reports based on file data.

File System

- Consider an example of a student's file system
  The student file will contain information regarding the student (i.e. roll no, student name, course etc.)

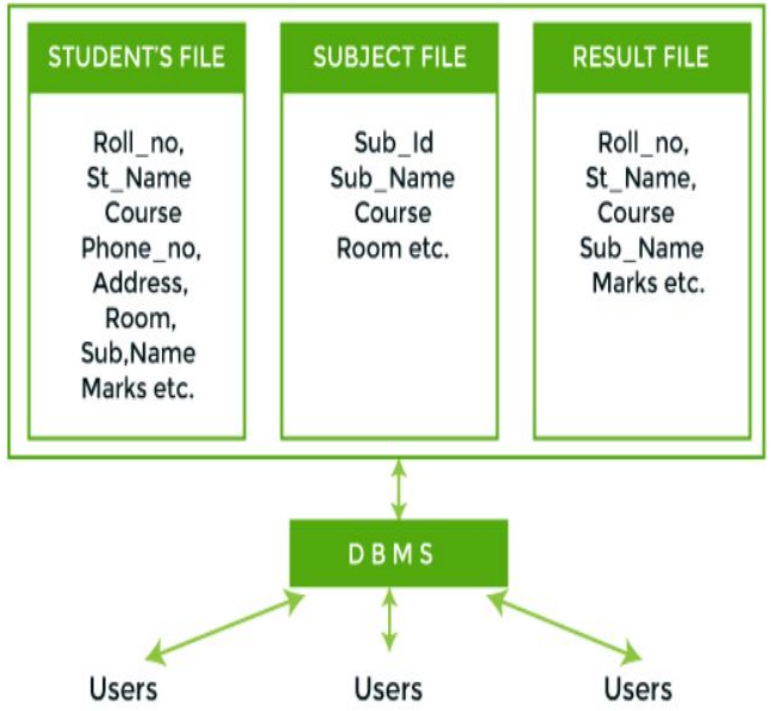


- Similarly, we have a subject file that contains information about the subject and the result file which contains the information regarding the result.

- Some fields are duplicated in more than one file, which leads to **data redundancy**. So to overcome this problem, we need to create a centralized system, i.e. DBMS approach

DBMS File System

DBMS



- A database approach is a well-organized collection of data that are related in a meaningful way which can be accessed by different users but stored only once in a system.
- The various operations performed by the DBMS system are: Insertion, deletion, selection, sorting etc.
- In the above figure, duplication of data is reduced due to centralization of data.

Difference between File system & DBMS

File System	Database Management System
1. File system is a software that manages and organizes the data files in a computer system.	1. DBMS is a software that is used for accessing , creating and managing the databases.
2. The File system provides the details of data representation and storage of data.	2. DBMS gives an abstract view of data that hides the details.
3. Redundant data can be present in a file system.	3. In DBMS there is no redundant data.
4.There is no efficient query processing in File system.	4. DBMS contains efficient query processing (SQL).
5.Backup and recovery of data is not efficient, because it is not possible to recover the lost data.	5. DBMS provides backup and recovery of data even if it is lost.

Difference between File system & DBMS

File System	Database Management System
6. Provide less security.	6. DBMS has more security mechanism as compared to file system.
7. Less complex	7. Complex
8. The file system does not have a crash recovery mechanism. i.e. if the system crashes while entering some data, than the content of the file will lost.	8. DBMS provides a crash recovery mechanism i.e. DBMS protects the user from the system failure
9.In the file system, concurrent access has many problems.	9. DBMS takes care of concurrent access of data using some form of locking mechanism.
10. File system is appropriate to handle data of a small scale organization or individual user.	10. DBMS is suitable for medium to large organization or multiple users.



Schema and State

1. Database Schema

❑ Database Schema

- The **description** of a database.
- Specified during database design and is not expected to change frequently.
- Includes descriptions of the database structure, data types, and the constraints on the database.

❑ Schema Diagram

- An **illustrative** display of (most aspects of) a database schema.

STUDENT

Name	Student_number	Class	Major

COURSE

Course_name	Course_number	Credit_hours	Department

PREREQUISITE

Course_number	Prerequisite_number

SECTION

Section_identifier	Course_number	Semester	Year	Instructor

GRADE_REPORT

Student_number	Section_identifier	Grade

2. Database State

- The actual data stored in a database at a **particular moment in time**. This includes the collection of all the data in the database.
- Also called database instance (or occurrence or snapshot).
- Refers to the **content** of a database at a moment in time.

Example of Database State

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Database Schema vs. Database State

- The database schema changes very infrequently.
- The database state changes every time the database is updated.
- Schema is also called **intension**.
- State is also called **extension**.

Database Users

❑ Database administrators

- In any organization where many people use the same resources, there is a need for a chief administrator to oversee and manage these resources.
- In a database environment, the primary resource is the database itself, and the secondary resource is the DBMS and related software.
- Database administrator (DBA) is responsible for authorizing access to the database, coordinating and monitoring its use, acquiring software and hardware resources as needed, controlling its use and monitoring efficiency of operations.
- The DBA is accountable for problems such as security breaches and poor system response time.

❑ Database designers

- Database designers are responsible for identifying the data to be stored in the database and for choosing appropriate structures to represent and store this data.
- Responsible to define the content, the structure, the constraints, and functions or transactions against the database. They must communicate with the end-users and understand their needs.

Database Users

- End-users
  - End users are the people whose jobs require access to the database for querying, updating, and generating reports; the database primarily exists for their use.
  - They use the data for queries, reports and some of them update the database content.

Three-Tier Architecture

Three-Schema Architecture

- The goal of the three-schema architecture is to separate user applications from the physical database.
- Defines DBMS schemas at **three** levels:
  - **Internal schema(or Physical Schema):** it describes physical storage structures and access paths (e.g. indexes).
  - Also Known as Low-level.
  - In simple terms, it tells "How data physically stored in DB".

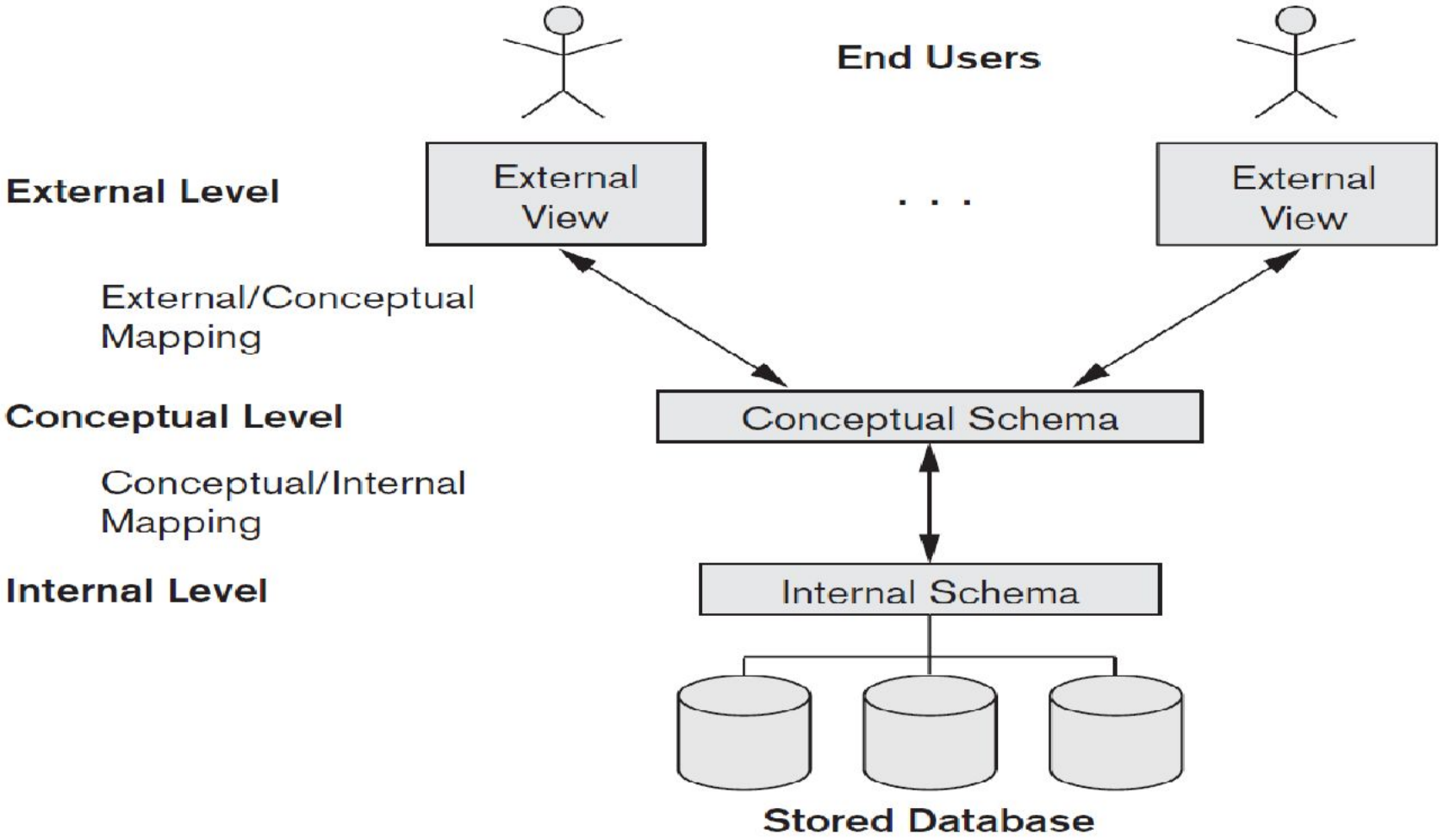
Three-Schema Architecture

- **Conceptual schema** at the conceptual level to describe the structure and constraints for the whole database for a community of users.
  - hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints.
  - In simple terms, it describes "What data exists in the Database".

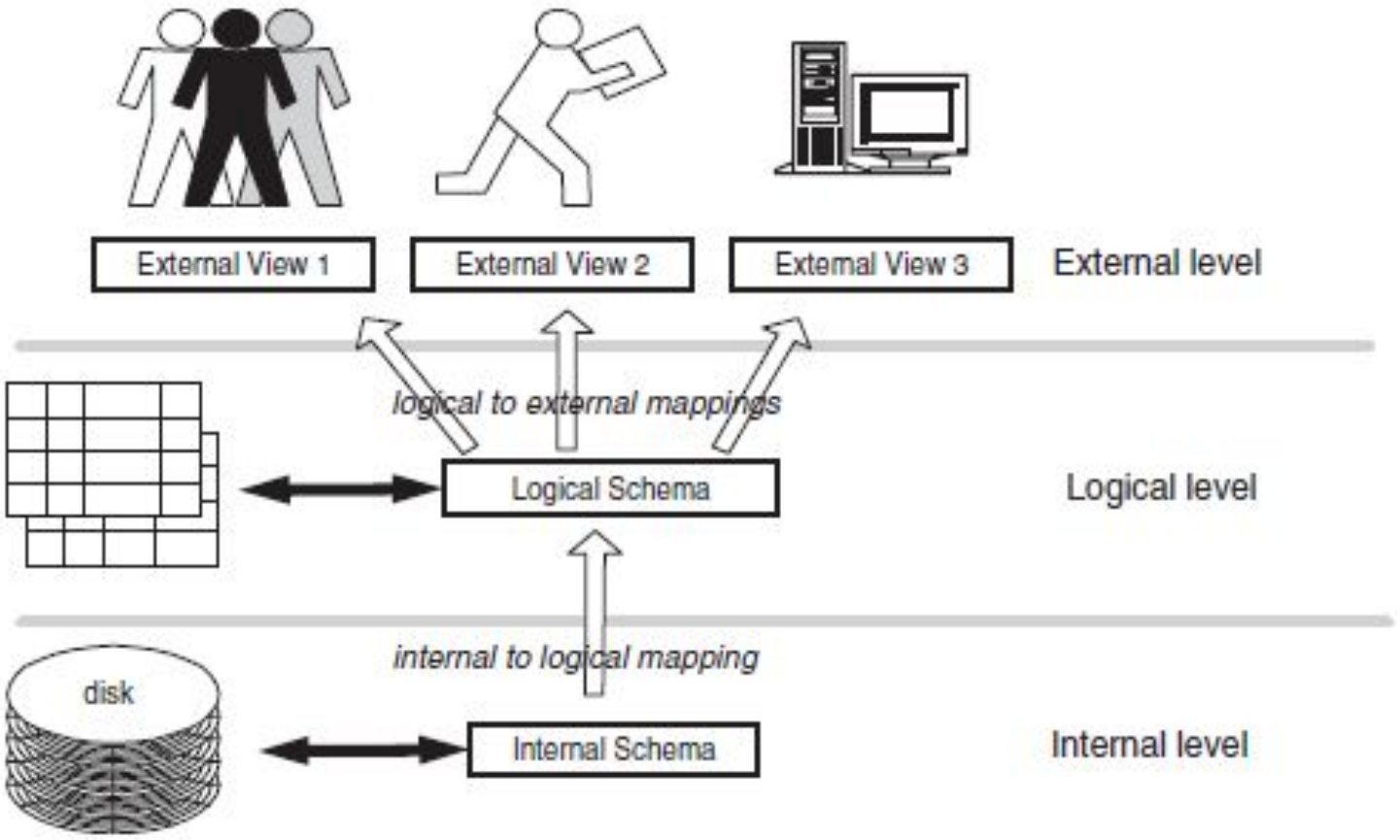
- **External schemas** at the external level to describe the various user views
  - Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group.

Three-Schema Architecture

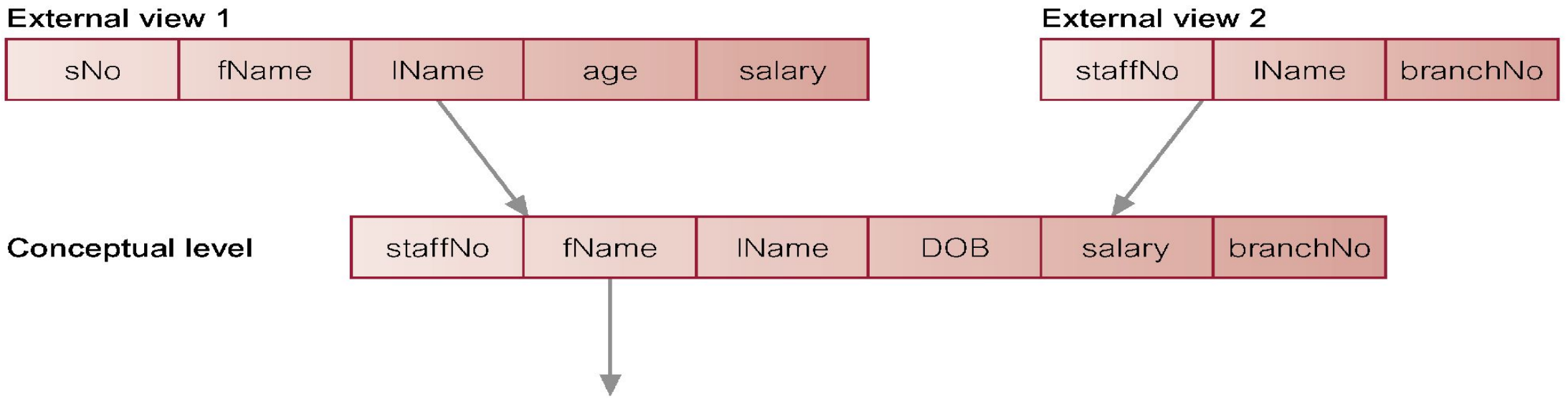
- The processes of transforming requests and results between levels are called **mappings**.



Three-Schema Architecture



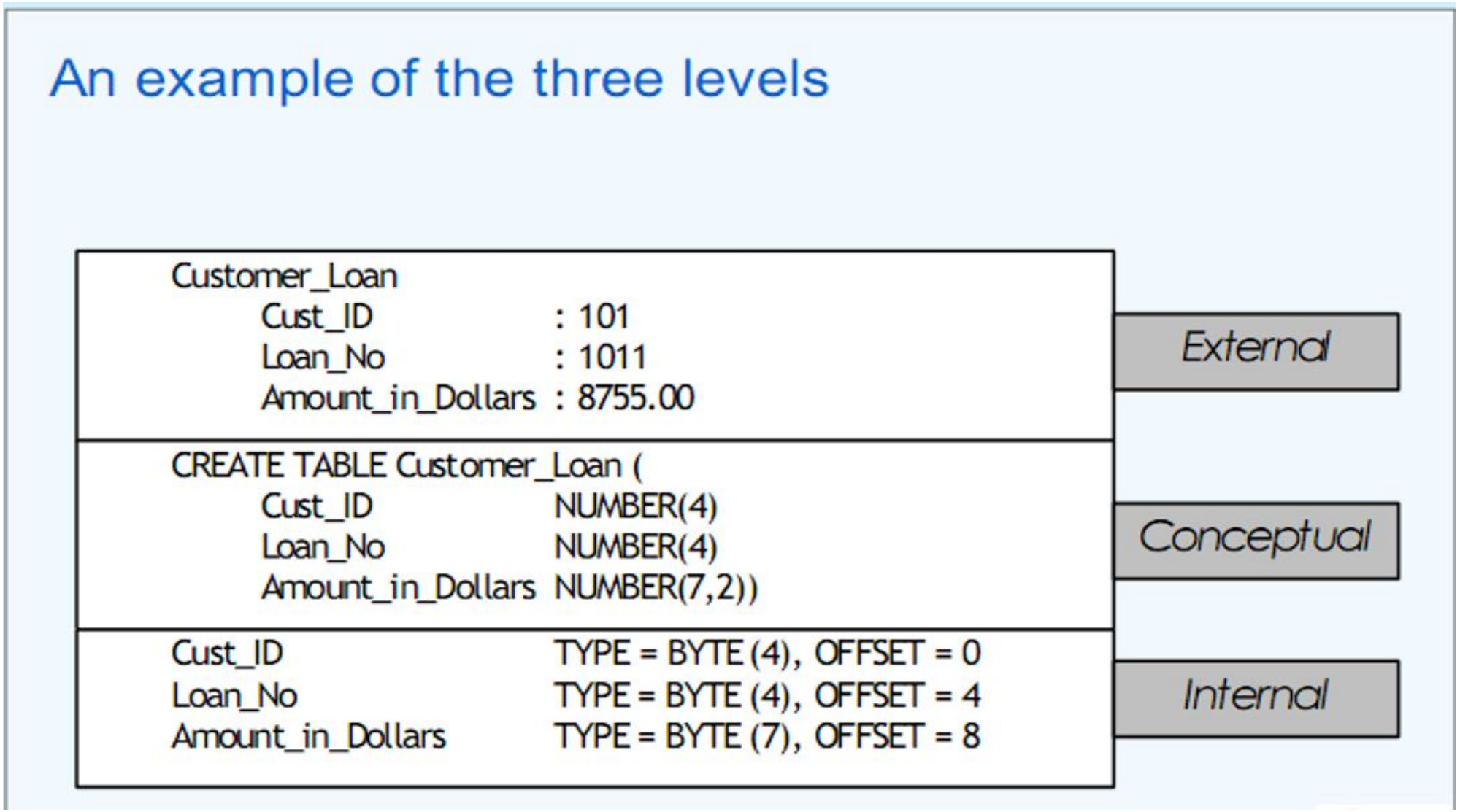
Three-Schema Architecture



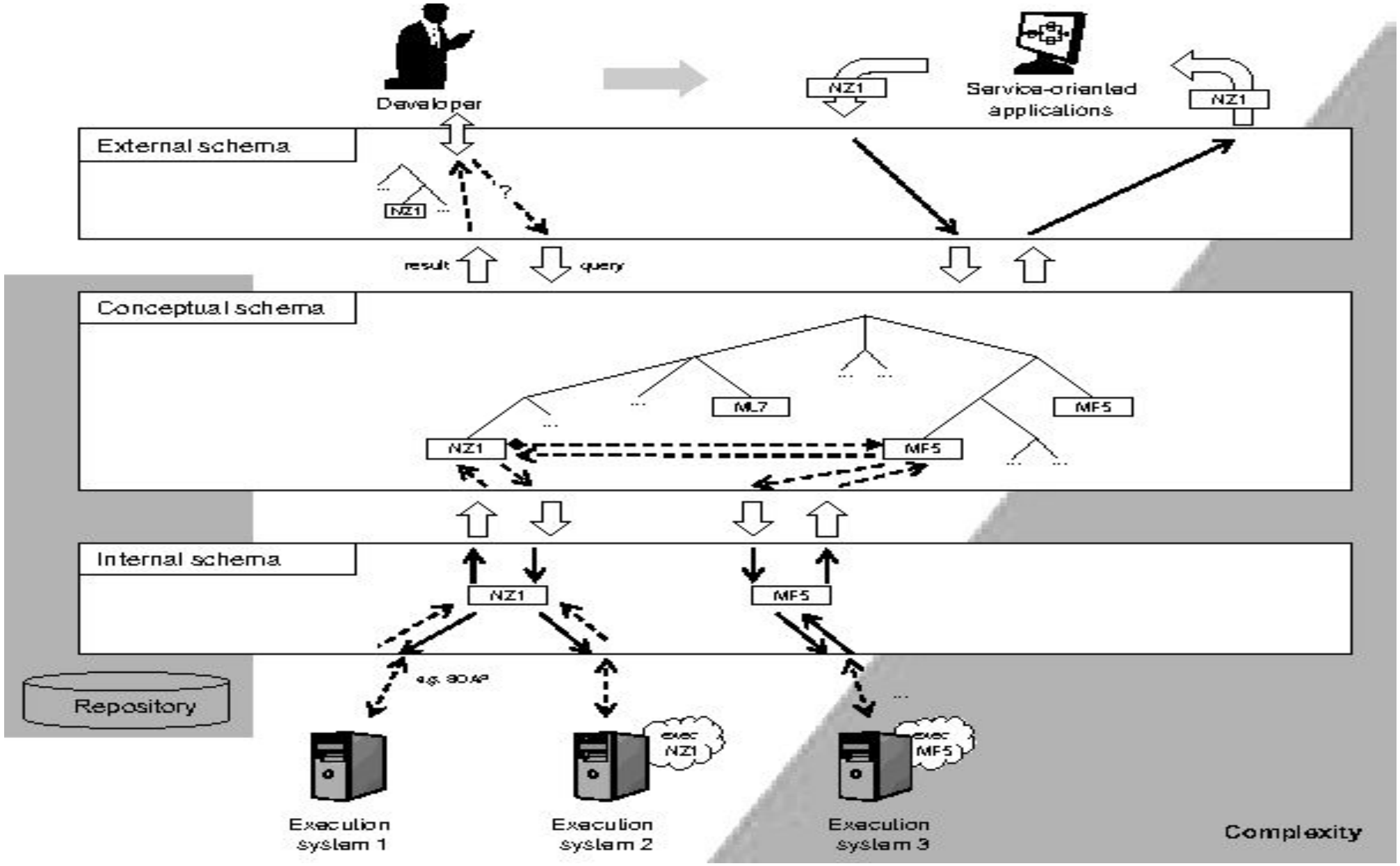
Internal level

```
struct STAFF {
        int staffNo;
        int branchNo;
        char fName [15];
        char lName [15];
        struct date dateOf Birth;
        float salary;
        struct STAFF *next;                    /* pointer to next Staff record */
};
index staffNo; index branchNo;                 /* define indexes for staff */
```

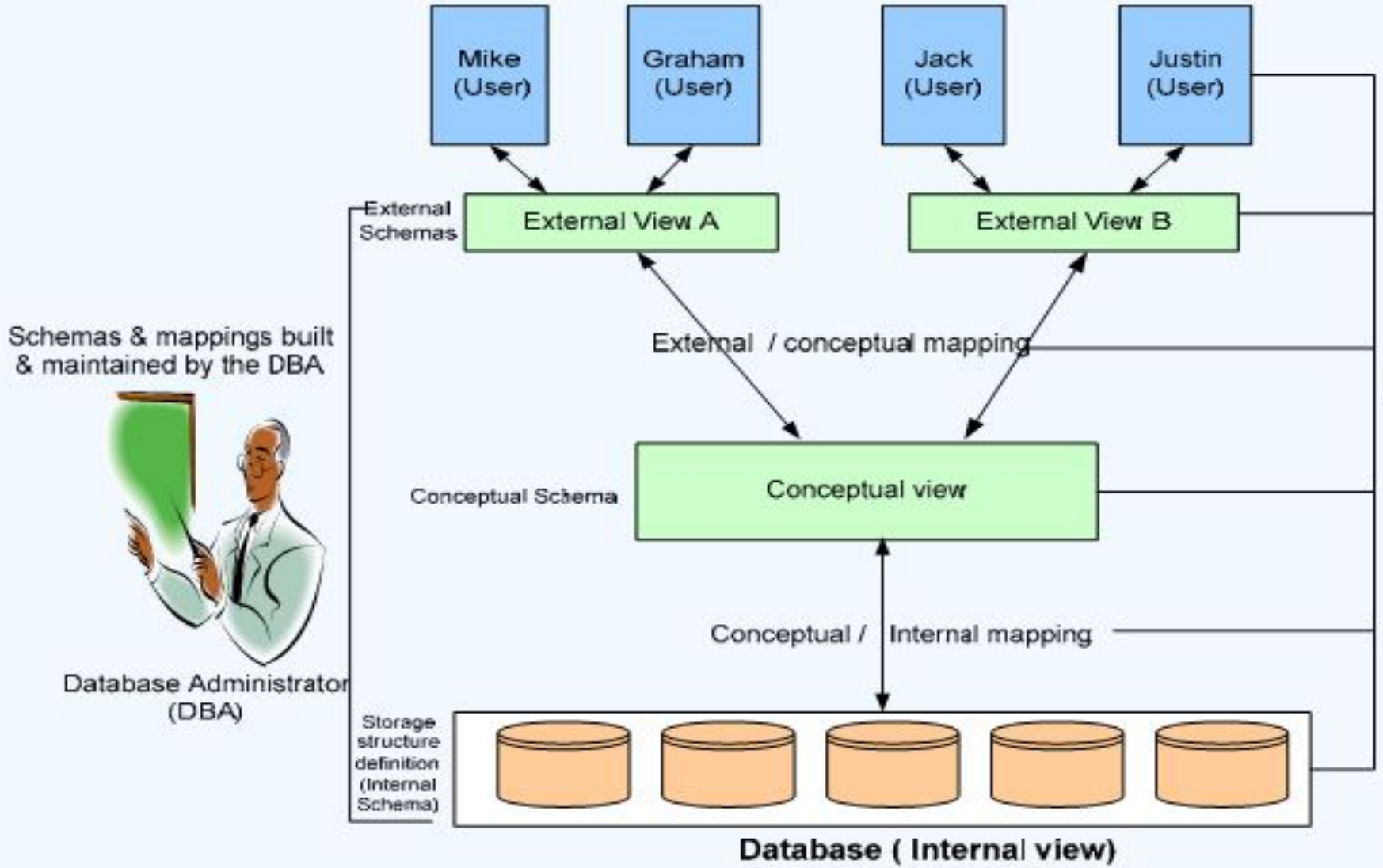
Three-Schema Architecture



Three-Schema Architecture



Detailed System Architecture



Data Independence

Data Independence

- It is defined as **a property** of a DBMS that helps to change the schema at one level of a database system without having to change the schema at the next higher level.

- **Types of Data Independence**
  - Physical data independence
  - Logical data independence

Data Independence

❑ 1. Physical data independence

- With Physical data independence, you can easily change the physical storage structures or devices without an effect on the conceptual schema.
- Any change done would be absorbed by the mapping between the conceptual and internal levels.

Data Independence

- **Examples of changes under Physical Data Independence:** Due to Physical independence, any of the below change will not affect the conceptual layer.

  - Using a new storage device like Hard Drive or Magnetic Tapes
  - Modifying the file organization technique in the Database
  - Switching to different data structures.
  - Changing the access method.
  - Modifying indexes.
  - Changes to compression techniques or hashing algorithms.
  - Change of Location of Database from say C drive to D Drive

Data Independence

- **2.** Logical data independence

  - Logical Data Independence is the ability to change the conceptual scheme without changing
    - External views
    - External API or programs
  - We may change the conceptual schema to expand the database (by adding records), to change the constraints, or to reduce the database.
  - For example: for user1 application program is different, for user2 application program is different, and so on.

- Logical data independence is achieved by implementing **VIEWS**. Views are virtual table.

Data Independence

- **Examples of changes under Logical Data Independence:** Due to Logical independence, any of the below change will not affect the external layer.
  - Add/Modify/Delete a new attribute, entity or relationship is possible without a rewrite of existing application programs
  - Merging two records into one
  - Breaking an existing record into two or more records

Difference between Physical and Logical Data Independence

Logica Data Independence	Physical Data Independence
Logical Data Independence is mainly concerned with the structure or changing the data definition.	Mainly concerned with the storage of the data.
Compared to Logic Physical independence it is difficult to achieve logical data independence.	Compared to Logical Independence it is easy to achieve physical data independence.
Concerned with conceptual schema	Concerned with internal schema
Example: Add/Modify/Delete a new attribute	Example: change in compression techniques, hashing algorithms, storage devices, etc

Classification of DBMSs

Classification of DBMSs

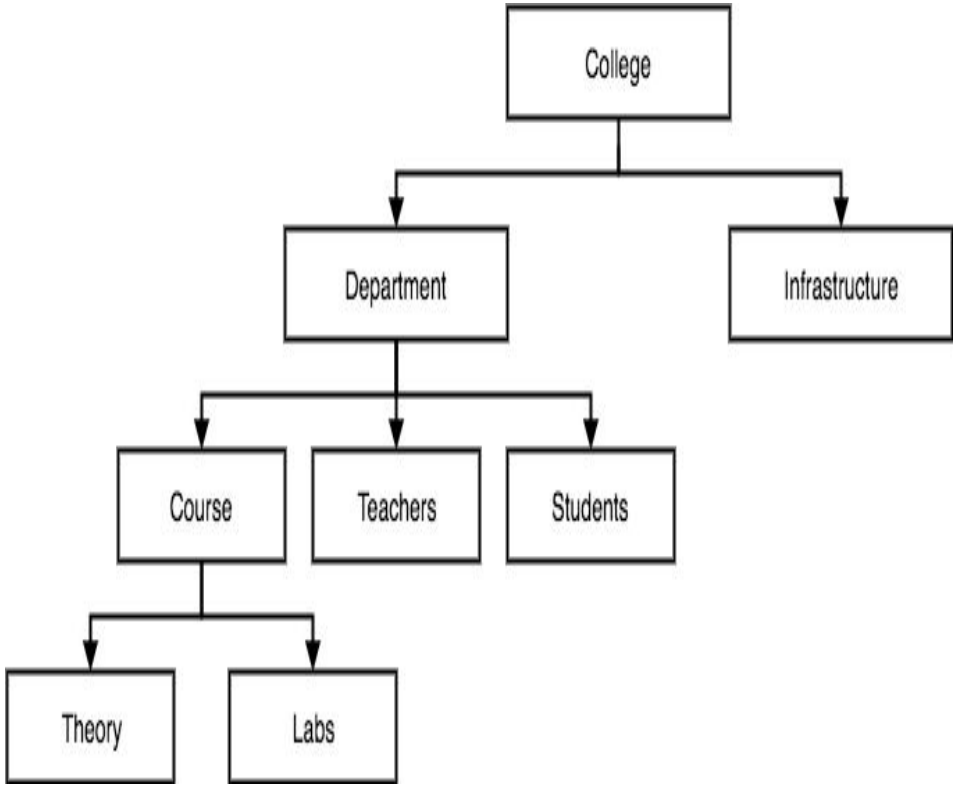
- Based on the data model used
  - Legacy: Hierarchical, Network.
  - Currently Used: Relational, Object-oriented, Object-relational
  - Recent Technologies: Key-value storage systems, NOSQL systems: document based, graph-based DBMSs etc.
- Based on Architecture
  - Single-user (typically used with personal computers) vs. multi-user (most DBMSs).
  - Centralized (uses a single computer with one database) vs. distributed (multiple computers, multiple DBs)

1. Based on Data Models

- A set of rules and standards that defines how the database organizes/store data is called database model.
- It also defines how users views the organization of data.

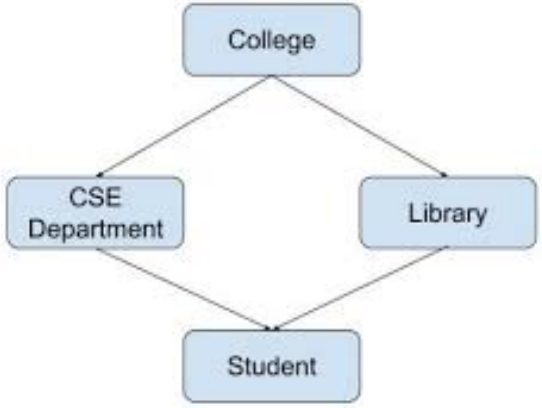
Hierarchical Model

- Hierarchical Database model is one of the oldest database models.
- The hierarchical model represents data as hierarchical tree structures. Each hierarchy represents a number of related records.
- One to many relationship.



Network Model

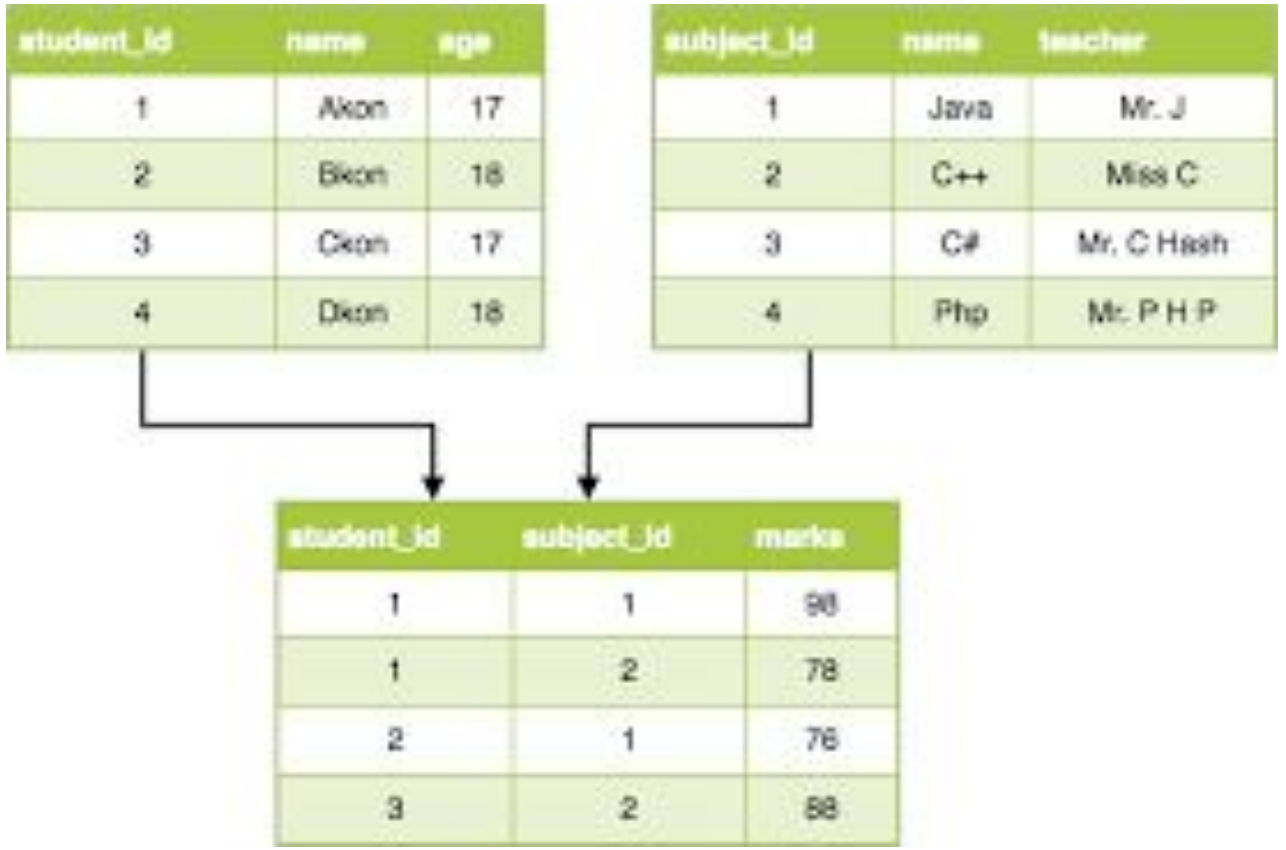
- It is the extension of Hierarchical Model.
- The Network model represents data as a graph.
- The main difference of the network model from the hierarchical model, is its ability to handle many to many (M:N) relations.



Network Model

Relational Model

- Relational model stores data in the form of Tables.
- Tables consist of rows and columns.
- Each column has specific data types and constraints.
- Tables are connected with other tables.



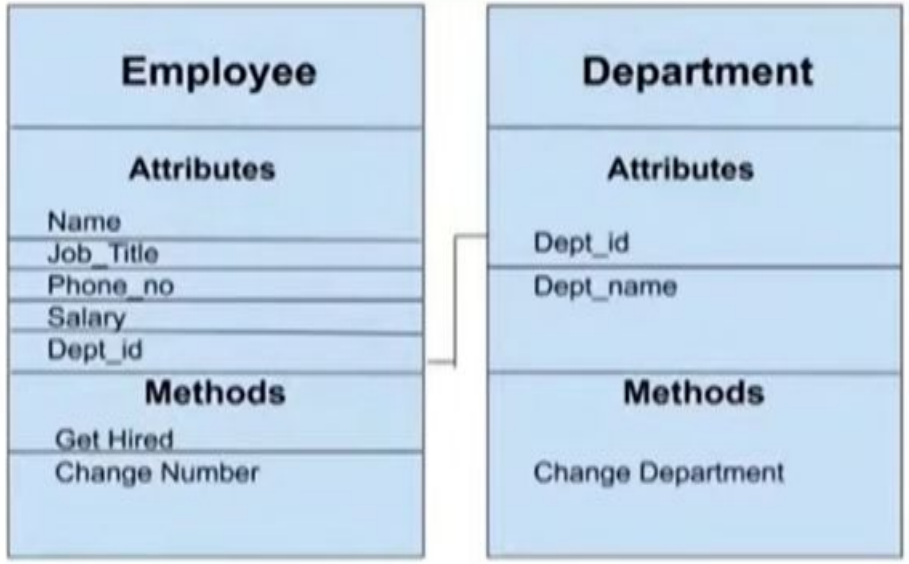
student_id	name	age
1	Akon	17
2	Bkon	18
3	Ckon	17
4	Dkon	18

subject_id	name	teacher
1	Java	Mr. J
2	C++	Miss C
3	C#	Mr. C Hash
4	Php	Mr. P H P

student_id	subject_id	marks
1	1	98
1	2	78
2	1	76
3	2	88

Object-oriented Data Models

- An Object oriented data model defines a database in terms of objects, their properties, and their operations.
- All the data and relationships of each object are combined as a single unit.
- Objects with the same structure and behaviour belong to a **class**.
- The operations of each class are specified in terms of predefined procedures called **methods**.



Object_Oriented_Model

Object-Relational Models

- Relational DBMSs have been extending their models to incorporate object database concepts.
- These systems are referred to as object-relational or **extended relational systems.**
- An Object relational model is a combination of a Object oriented database model and a Relational database model. So, it supports objects, classes, inheritance etc. just like Object Oriented models and has support for data types, tabular structures etc. like Relational data model.

- One of the major goals of Object relational data model is to close the gap between relational databases and the object oriented practises frequently used in many programming languages such as C++, C#, Java etc.
- The problem with this model is that this can get complex and difficult to handle. So, proper understanding of this model is required.

- Relational model = ER diagrams
- Object-Relational = EER diagrams

XML model

- Extensible Markup language (XML) is widely used for representing the semi-structured data.
- Semi structured model is an evolved form of the relational model.
- The XML model has emerged as a standard for exchanging data over the Web.
- XML uses hierarchical tree structures.
- It combines database concepts with concepts from document representation models.
- Data is represented as elements with the use of tags, data can be nested to create complex hierarchical structures.

XML

- <student 1>
  <Roll No.>.....</Roll No.>
  <Name>.....</Name>
  <Class>.....</Class>
  <Age>.....</Age>

</student 1>

<student 2>
  <Name>.....</Name>
  <Class>.....</Class>
  <Age>.....</Age>

</student 2>

2. Based on Architecture

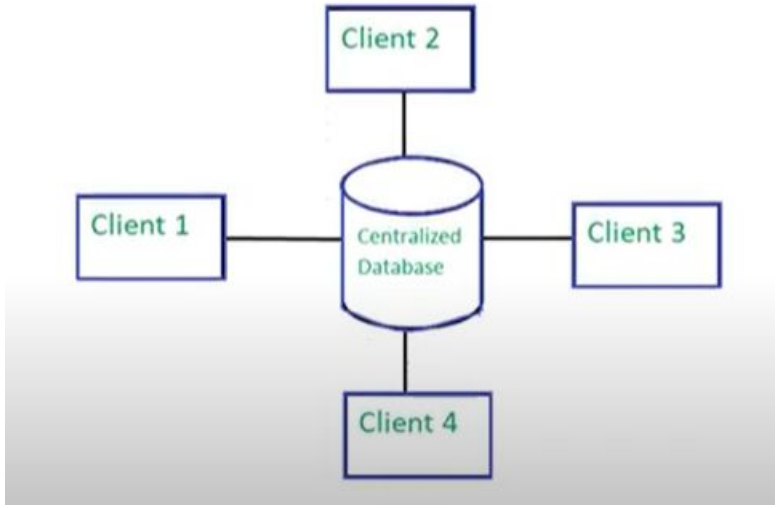
1. Centralised Database

- A database that is stored, located and maintained at a single location this centralized database is accessed via an internet connection.

Advantages

- it is less costly.
- it is easy to manage and update.
- very minimal data redundancy because all the

data is stored at a single place.



Centralised Database

Disadvantages

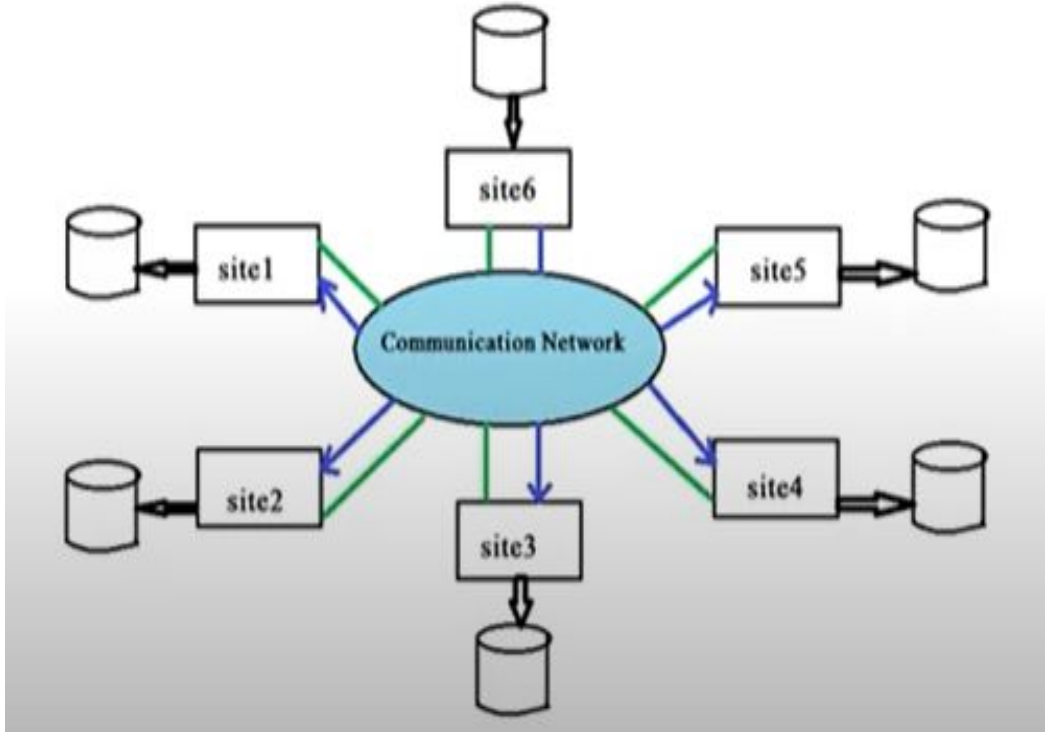
- Performance issues(multiple users access the database at the same time).
- Due to single failure data will be lost and user cant access the database
- Highly dependent on network.

2. Distributed Database

- It is an integrated collection of database that is physically distributed across sites in a computer network.
- that appears to users as a single database.

Advantage

- If one database fails then users can access the other database files.
- This databases is more secure as compared to centralised database.



Distributed Database

- More reliable -Due to single failure entire data will not lost.
- Data traffic is less.
- Performance- No bottlenecks as the load is spread over multiple server.

Disadvantages

- Data integrity concerns- need to update data in multiple sites.
- Time for synchronization between sites.
- This database is costly.
- It is difficult to maintain and update due to its complexity.

Database Languages

Database Languages

- A database system provides:
  - **data-definition language (DDL)** to specify the database schema.
  - **data-manipulation language (DML)** to express database queries and updates.
  - **data-control language (DCL)** : it controls the access to the data.

DDL: Create, alter, drop, rename

DML: insert, update, delete, select

DCL: revoke, grant

TCL: Rollback, Commit, Savepoint

DQL: Where, from, Select etc.

Data Definition Language (DDL)

- The output of the DDL is placed in the **data dictionary**, which contains **metadata**—that is, data about data.
- DDL compiler generates a set of table templates stored in a data dictionary.
- **Example:**

```
create table instructor (
    ID          char(5),
    name        varchar(20),
    dept_name   varchar(20),
    salary      double(8,2));
```

- Data dictionary contains metadata (i.e., data about data)
  - Database schema
  - Integrity constraints
    - Primary key (ID uniquely identifies instructors)
  - Authorization
    - Who can access what

Data Manipulation Language (DML)

- Language for accessing and updating the data organized by the appropriate data model. The types of access are:
  - Retrieval of information stored in the database
  - Insertion of new information into the database
  - Deletion of information from the database
  - Modification of information stored in the database.
- There are basically two types of data-manipulation language:
  - **Procedural DMLs -** require a user to specify *what* data are needed and *how* to get those data.
  - **Declarative DMLs (Nonprocedural DMLs) -** require a user to specify *what* data are needed *without* specifying how to get those data.
- Declarative DMLs are usually easier to learn and use than are procedural DMLs.

Data Manipulation Language (DML)

- A **query** is a statement requesting the retrieval of information.
- The portion of a DML that involves information retrieval is called a **query language**.

SQL

- SQL is procedural. A query takes as input one or several tables and always returns a single table.
- Example to find all instructors in Comp. Sci. dept:

```
select name
from instructor
where dept_name = 'Comp. Sci.';
```

- The query specifies that those rows from the table *instructor* where the *dept_name* is Comp. Sci. must be retrieved, and the name attribute of these rows must be displayed. The result of executing this query is a table with a single column labeled *name* and a set of rows, each of which contains the name of an instructor whose *dept_name* is Comp. Sci.

SQL

- Queries may involve information from more than one table.
- The following query finds the instructor ID and department name of all instructors associated with a department with a budget of more than Rs. 95,000.

```
select instructor.ID, department.dept_name
from instructor, department
where instructor.dept_name= department.dept_name and
department.budget > 95000;
```

- SQL is NOT a Turing machine equivalent language.
- To be able to compute complex functions SQL is usually embedded in some higher-level language.

DQL - Data Query Language

- DQL statements are used for performing queries on the data within schema objects.

Command	Description	Syntax
SELECT	It is used to retrieve data from the database	SELECT column1, column2, ...FROM table_name WHERE condition;
FROM	Indicates the **table(s)** from which to retrieve data.	SELECT column1 FROM table_name;
WHERE	Filters rows **before** any grouping or aggregation	SELECT column1 FROM table_name WHERE condition;
GROUP BY	Groups rows that have the same values in specified columns.	SELECT column1, AGG_FUNCTION(column2) FROM table_name GROUP BY column1;
HAVING	Filters the results of GROUP BY	SELECT column1, AGG_FUNCTION(column2) FROM table_name GROUP BY column1 HAVING condition;
DISTINCT	Removes **duplicate rows** from the result set	SELECT DISTINCT column1, column2, ...FROM table_name;
ORDER BY	Sorts the result set by one or more columns	SELECT column1 FROM table_name ORDER BY column1 [ASC | DESC];
LIMIT	By default, it sorts in **ascending order** unless specified as DESC	SELECT * FROM table_name LIMIT number;

DCL - Data Control Language

- DCL deal with the rights, permissions and other controls of the database system eg. GRANT and REVOKE.

Command	Description	Syntax
GRANT	Assigns new privileges to a user account, allowing access to specific database objects, actions, or functions.	GRANT privilege_type [(column_list)] ON [object_type] object_name TO user [WITH GRANT OPTION];
REVOKE	Removes previously granted privileges from a user account, taking away their access to certain database objects or actions.	REVOKE [GRANT OPTION FOR] privilege_type [(column_list)] ON [object_type] object_name FROM user [CASCADE];

- Example: GRANT SELECT, UPDATE ON employees TO user_name;

TCL - Transaction Control Language

- Transactions group a set of tasks into a single execution unit.
- Each transaction begins with a specific task and ends when all the tasks in the group are successfully completed. If any of the tasks fail, the transaction fails.

Command	Description	Syntax
BEGIN TRANSACTION	Starts a new transaction	BEGIN TRANSACTION [transaction_name];
COMMIT	Saves all changes made during the transaction	COMMIT;
ROLLBACK	Undoes all changes made during the transaction	ROLLBACK;
SAVEPOINT	Creates a savepoint within the current transaction	SAVEPOINT savepoint_name;

Example:

```
BEGIN TRANSACTION;
UPDATE employees SET department = 'Marketing' WHERE department = 'Sales';
SAVEPOINT before_update;
UPDATE employees SET department = 'IT' WHERE department = 'HR';
ROLLBACK TO SAVEPOINT before_update;
COMMIT;
```

Database Development Life Cycle

The different phases of database development life cycle (DDLC) in the Database Management System (DBMS) are explained below −

- Requirement analysis.
- Database design.
- Evaluation and selection.
- Logical database design.
- Physical database design.
- Implementation.
- Data loading.
- Testing and performance tuning.
- Operation.
- Maintenance

1. Requirement Analysis

- The most important step in implementing a database system is to find out what is needed i.e what type of a database is required for the business organization, daily volume of data, how much data needs to be stored in the master files etc.
- In order to collect all this information, a database analyst spends a lot of time within the business organization talking to people, end users and getting acquainted with the day-to-day process.

2. Database Design

- In this phase the database designers will make a decision on the database model that perfectly suits the organization's requirement. The database designers will study the documents prepared by the analysis in the requirement analysis stage and then start development of a system model that fulfils the needs.

3. Evaluation and selection

- In this phase, we evaluate the diverse database management systems and choose the one which perfectly suits the requirements of the organization.
- In order to identify the best performing database, end users should be involved.

4. Physical Database Design

- This phase selects and characterizes the data storage and data access of the database.
- The data storage depends on the type of devices supported by the hardware, the data access methods.
- Physical design is very vital because of bad design which results in poor performance.

5. Logical database design

- Once the evaluation and selection phase is completed successfully, the next step is logical database design.
- This design is translated into internal model which includes mapping of all objects i.e design of tables, indexes, views, transaction, access privileges etc.,

6. Implementation

- Database implementation needs the formation of special storage related constructs.
- These constructs consist of storage groups, table spaces, data files, tables etc.

7. Data Loading

- Once the database has been created, the data must be loaded into the database.
- The data required to be converted, if the loaded date is in a different format.

8. Operations

- In this phase, the database is accessed by the end users and application programs.
- This stage includes adding of new data, modifying existing data and deletion of absolute data.
- This phase provides useful information and helps management to make a business decision.

9. Maintenance

- It is one of the ongoing phases in DDLC.
- The major tasks included are database backup and recovery, access management, hardware maintenance etc.



Thank You